Classes & Objects in Java

A complete beginner guide — from zero to writing your own classes

■ Table of Contents

1. What is a Class? The Blueprint Idea

Before writing any code, you need to understand what a class *really is* — not just the definition, but the **mental model** behind it.

> **■ Analogy: A House Blueprint**
>
> Imagine an architect draws a blueprint for a house. That blueprint describes: how many rooms, where the windows go, what the dimensions are. The blueprint itself is **not a house** — it's just a plan. But from that one blueprint, you can build **many houses**. Each house is real, has its own address, its own colour, its own furniture. In Java: **Class = Blueprint. Object = The actual house built from it.**

The Formal Definition

A **class** is a user-defined data type that acts as a template. It defines two things for every object created from it:

- **State** — the data it holds (called **fields** or instance variables). Example: a circle's radius, x-coordinate, y-coordinate.
- **Behaviour** — what it can do (called **methods**). Example: calculate area, move to a new position.

> **■ Another Analogy: A Coffee Recipe**
>
> The **recipe** for making coffee is a class. It says: use 2 spoons of coffee, 200ml of water, heat to 90°C. The **actual cup of coffee** sitting on your desk is the object — it was made using the recipe. Your friend's cup is a different object, made from the same recipe, but it's its own separate cup.

Why Not Just Use Variables?

Without classes, if you wanted to track 100 students, you'd need 100 separate variables for name, 100 for age, 100 for GPA — a nightmare to manage. With a class, you define *Student* once and create 100 clean, organised Student objects.

> **■ Key Insight**
>
> A class itself does nothing and uses no memory for data. Only when you create an **object** from the class does Java actually allocate memory and bring it to life.

2. Anatomy of a Java Class — Fields & Methods

Every Java class has two main ingredients: **fields** (data) and **methods** (actions). Here is the general structure:

```
// Java
public class ClassName {
```

```
// ■■ FIELDS (data the object holds) ■■■■■■■■■■■■■■■■■■■■
dataType fieldName;
dataType fieldName2;
// ■■ METHODS (things the object can do) ■■■■■■■■■■■■■■■■
returnType methodName(parameter-list) {
// method body
}
}
```

① Fields — The Object's Data

Fields are variables declared **inside the class but outside any method**. Each object gets its own *copy* of every field — that's why they're called **instance variables**.

```
// Java
public class Circle {
public double x; // x-coordinate of centre
public double y; // y-coordinate of centre
public double r; // radius
// Every Circle object will have its OWN x, y, r values
}
```

② Methods — The Object's Behaviour

Methods are functions defined inside a class. They describe what the object *can do*. A method with no data fields is useless — objects created from it can't respond to anything.

```
// Java
public class Circle {
public double x, y; // centre
public double r; // radius
// Method 1: calculate circumference
public double circumference() {
return 2 * 3.14 * r;
}
// Method 2: calculate area
public double area() {
return 3.14 * r * r;
}
}
```

Parts of a Method — Explained

Part	Example	What it means
Return type	double	What the method sends back. Use void if it returns nothing.
Method name	area	What you call it. Use descriptive verbs.
Parameters	(int x, int y)	Input values the method needs. Can be empty ().

Method body	{ return 3.14*r*r; }	The actual code that runs.

■ Class = Data + Functions Together

Before OOP, data and functions were separate — anyone could change any data. A class **bundles them together**, so the data is protected and only the class's own methods should work on it. This is **encapsulation**.

3. Creating Objects — Bringing Classes to Life

Defining a class only creates the blueprint. To actually **use** a class, you must create an **object** (also called an **instance**) from it.

> ■ **Analogy: Cookie Cutter**
>
> A class is like a **cookie cutter**. The cutter defines the shape. Each cookie you press out is an **object** — it has the same shape (same fields, same methods) but it's a separate, independent cookie. You can make as many cookies as you want from one cutter.

Step 1 — Declare a Reference Variable

This just creates a variable that *can* point to a Circle object. At this point, no object exists yet — it points to nothing (null).

```
// Java
Circle aCircle; // declares a reference — no object yet!
// aCircle currently = null (points to nothing)
```

Step 2 — Create the Object with `new`

The **new** keyword is what actually creates the object in memory. Java allocates space, initialises the fields, and returns a reference to the new object.

```
// Java
aCircle = new Circle(); // NOW an object exists in memory!
// aCircle now points to a real Circle object
// Or combine both steps in one line:
Circle bCircle = new Circle();
```

You Can Create Many Objects from One Class

```
// Java
Circle c1 = new Circle(); // Object 1 — has its OWN x, y, r
Circle c2 = new Circle(); // Object 2 — completely separate
Circle c3 = new Circle(); // Object 3 — also independent
c1.r = 5.0; // c1's radius is 5
c2.r = 10.0; // c2's radius is 10 — c1 is NOT affected
c3.r = 2.5; // c3's radius is 2.5
```

What Happens with Reference Assignment?

When you assign one reference to another, **you are NOT copying the object**. Both variables end up pointing to the *same* object.

```
// Java
Circle aCircle = new Circle(); // aCircle → Object P
Circle bCircle = new Circle(); // bCircle → Object Q
```

```
bCircle = aCircle; // NOW both point to Object P!
// Object Q is now unreferenced → garbage collected
// Any change via aCircle is visible through bCircle too:
aCircle.r = 99.0;
System.out.println(bCircle.r); // Output: 99.0 (same object!)
```

■■ Reference vs Object

Circle aCircle; — this is a reference (like a label on a string). No actual circle exists yet. **aCircle = new Circle();** — this creates the real object and ties the label to it. Always use **new** to create objects!

4. How to Access Data and Call Methods

Once you have an object, you interact with it using the **dot operator (.)**. This is the only member access operator in Java (unlike C++ which has . and ->).

```
// Java
// Syntax:
objectName.fieldName // access a field
objectName.methodName() // call a method
objectName.methodName(args) // call a method with arguments
```

Accessing Fields

```
// Java
Circle aCircle = new Circle();
// Set field values using dot operator
aCircle.x = 10.0; // set x-coordinate
aCircle.y = 20.0; // set y-coordinate
aCircle.r = 5.0; // set radius
// Read field values
System.out.println(aCircle.r); // Output: 5.0
```

Calling Methods

```
// Java
Circle aCircle = new Circle();
aCircle.r = 5.0;
// Call methods — the method uses the object's OWN data
double area = aCircle.area(); // uses aCircle.r
double circumf = aCircle.circumference(); // uses aCircle.r
System.out.println("Area = " + area); // 78.5
System.out.println("Circumference = " + circumf); // 31.4
```

Complete Working Example

```java
// Java
class MyMain {
public static void main(String[] args) {
Circle aCircle; // Step 1: declare reference
aCircle = new Circle(); // Step 2: create object
aCircle.x = 10; // Step 3: set data
aCircle.y = 20;
aCircle.r = 5;
double area = aCircle.area(); // Step 4: call method
double circf = aCircle.circumference(); // Step 4: call method
System.out.println("Radius = " + aCircle.r);
System.out.println("Area = " + area);
System.out.println("Circumference = " + circf);
}
}
// Output:
// Radius = 5.0
// Area = 78.5
// Circumference = 31.400000000000002
```

■ Analogy: Sending a Message

Calling a method on an object is like **sending a message** to it. When you write `aCircle.area()`, you're sending the message 'calculate your area' to the aCircle object. The object then uses its own data (r=5.0) to compute and reply with the answer 78.5.

5. References, null & Garbage Collection

Understanding how Java manages object memory saves you from confusing bugs.

What is a Reference?

A reference variable holds the **memory address** of an object — it's like a label on a piece of string that leads to the actual object. It is similar to a pointer in C, but safer because you **cannot** do pointer arithmetic or point to arbitrary memory.

> **■■ Analogy: A Label on a String**
>
> Imagine you tie a label to a balloon. The label says 'aCircle'. The balloon is the actual object in memory. If you cut the string (set aCircle = null), the balloon floats away — and Java's garbage collector pops it when it needs the space back.

The `null` Reference

A reference variable that doesn't point to any object yet holds the value **null**. Trying to use a null reference causes a **NullPointerException** — one of the most common runtime errors in Java.

```
// Java
Circle aCircle; // automatically = null (class-level)
aCircle = null; // explicitly set to null
// DANGER — this crashes with NullPointerException:
// double area = aCircle.area(); ← aCircle is null!
// Always check before using:
if (aCircle != null) {
double area = aCircle.area(); // safe
}
```

Automatic Garbage Collection

In C/C++, programmers must manually free memory. Java does this for you automatically through the **Garbage Collector (GC)**.

- When no reference points to an object, the object is **unreachable**.
- The GC runs periodically in the background.
- It finds unreachable objects and frees their memory automatically.

```
// Java
Circle aCircle = new Circle(); // aCircle → Object P
Circle bCircle = new Circle(); // bCircle → Object Q
bCircle = aCircle; // bCircle now points to Object P
// Object Q has NO references → eligible for GC
// Java will reclaim Q's memory automatically
```

> **■■ You Never Need to Delete Objects**

Unlike C++, Java has no **delete** keyword. Just stop pointing to an object and Java's garbage collector handles the cleanup. This eliminates memory leaks and dangling pointer bugs.

6. The `this` Keyword

Inside any method, the keyword **this** refers to *the current object* — the specific object whose method is being called right now.

■ Analogy: Referring to Yourself

When you say 'I am hungry', the word **I** refers to you specifically — not everyone in the room. In Java, **this** works the same way inside a method: it means 'the object running this method right now'.

Main Use Case: Resolving Name Conflicts

The most common use of `this` is when a method parameter has the **same name** as a field. Without `this`, Java gets confused about which one you mean:

```
// Java
public class Box {
double width;
double height;
double depth;
// Constructor where parameter names match field names
Box(double width, double height, double depth) {
// 'width' alone = the parameter (local variable)
// 'this.width' = the field belonging to this object
this.width = width; // ■ correct — field = parameter
this.height = height;
this.depth = depth;
}
}
```

Without `this` — A Bug!

```
// Java
Box(double width, double height, double depth) {
width = width; // ■ WRONG — just sets the parameter to itself
height = height; // ■ WRONG — the fields never get updated!
depth = depth; // ■ WRONG
}
// The fields remain 0 — this is called 'instance variable hiding'
```

Other Uses of `this`

- **this.fieldName** — explicitly refers to the object's field (most common use).

- **this.methodName()** — calls another method of the same object.
- **return this;** — returns the current object (used in method chaining).

7. Full Real-Life Example — Circle Class

Let's build the complete Circle class from scratch, step by step, using everything learned so far.

```
// Java
// ■■ Step 1: Define the class ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class Circle {
// Fields — every Circle object has its own copy of these
public double x; // x-coordinate of centre
public double y; // y-coordinate of centre
public double r; // radius
// Constructor — runs automatically when object is created
public Circle(double x, double y, double r) {
this.x = x; // 'this' distinguishes field from parameter
this.y = y;
this.r = r;
}
// Method 1: calculate area
public double area() {
return 3.14 * r * r;
}
// Method 2: calculate circumference
public double circumference() {
return 2 * 3.14 * r;
}
// Method 3: move the circle to a new position
public void move(int newX, int newY) {
this.x = newX;
this.y = newY;
}
// Method 4: print info about this circle
public void display() {
System.out.println("Circle at (" + x + "," + y + ") radius=" + r);
}
}
```

```
// Java
// ■■ Step 2: Use the class ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class Main {
public static void main(String[] args) {
// Create two separate Circle objects
Circle c1 = new Circle(0, 0, 5.0); // centre (0,0) radius 5
Circle c2 = new Circle(10, 20, 3.0); // centre (10,20) radius 3
c1.display(); // Circle at (0.0,0.0) radius=5.0
c2.display(); // Circle at (10.0,20.0) radius=3.0
```

```
System.out.println("c1 area = " + c1.area()); // 78.5
System.out.println("c2 area = " + c2.area()); // 28.26
c1.move(3, 4); // move c1 to (3,4)
c1.display(); // Circle at (3.0,4.0) radius=5.0
// c2 is unaffected — they are SEPARATE objects
c2.display(); // Circle at (10.0,20.0) radius=3.0
}
}
```

8. Full Real-Life Example — Student Record System

Here is a more relatable example. This shows how you would model a student in a university system — using **private** fields (proper encapsulation) and **getter/setter** methods.

```
// Java
public class Student {
// Private fields — can't be accessed directly from outside
private String name;
private int studentId;
private double gpa;
private String department;
// Constructor — initialise all fields when object is created
public Student(String name, int id, String dept) {
this.name = name;
this.studentId = id;
this.department = dept;
this.gpa = 0.0; // starts at 0
}
// Getters — controlled READ access
public String getName() { return name; }
public int getStudentId() { return studentId; }
public double getGpa() { return gpa; }
public String getDepartment() { return department; }
// Setter with validation — controlled WRITE access
public void setGpa(double gpa) {
if (gpa >= 0.0 && gpa <= 4.0) {
this.gpa = gpa;
} else {
System.out.println("Invalid GPA! Must be 0.0 - 4.0");
}
}
// A method that does something useful
public String getGrade() {
if (gpa >= 3.7) return "A+";
```

```
else if (gpa >= 3.3) return "A";
else if (gpa >= 3.0) return "A-";
else if (gpa >= 2.7) return "B+";
else return "Below B+";
}
public void display() {
System.out.println("--- Student Record ---");
System.out.println("Name : " + name);
System.out.println("ID : " + studentId);
System.out.println("Dept : " + department);
System.out.println("GPA : " + gpa + " (" + getGrade() + ")");
}
}
```

```
// Java
public class Main {
public static void main(String[] args) {
Student s1 = new Student("Alice", 101, "CSE");
Student s2 = new Student("Bob", 102, "EEE");
s1.setGpa(3.8); // valid
s2.setGpa(9.9); // ■ prints: Invalid GPA! Must be 0.0 - 4.0
s2.setGpa(2.9); // valid
s1.display();
// --- Student Record ---
// Name : Alice
// ID : 101
// Dept : CSE
// GPA : 3.8 (A+)
s2.display();
// --- Student Record ---
// Name : Bob
// ID : 102
// Dept : EEE
// GPA : 2.9 (B+)
}
}
```

■ Why Private Fields + Getters/Setters?

If **gpa** were public, any code could write `s1.gpa = 99.9` with no checks. By making it private and using a setter, you guarantee the GPA is always valid — the class **controls its own data**. This is encapsulation in action.

9. Common Mistakes to Avoid

These mistakes are made by almost every beginner. Study them carefully.

■ Wrong	■ Correct
Circle c; double a = c.area(); // NullPointerException! c was never created.	Circle c = new Circle(); double a = c.area(); // Always use 'new' before using an object.
Box(double width) { width = width; // sets parameter to itself }	Box(double width) { this.width = width; // field = parameter }
Circle a = new Circle(); Circle b = a; b.r = 99; // a.r is now 99 too! // You copied the reference, not the object.	// Understand that b = a makes both point // to the SAME object. Changes via b // affect a as well.
public double r; // public field c.r = -500; // no validation possible!	private double r; // private field public void setR(double r) { if (r > 0) this.r = r; }
void move(int xCoord) { x = xCoord; // works only if no local 'x' }	void move(int x) { this.x = x; // always explicit // 'this.x' = field, 'x' = parameter }

10. Quick Reference Card

Question	Answer
What is a class?	A blueprint/template that defines the state (fields) and behaviour (methods) of objects.
What is an object?	A real instance of a class, created with the new keyword. Has its own copy of all fields.
What is a field?	A variable declared inside a class but outside methods. Each object has its own copy.
What is a method?	A function inside a class. Defines what the object can do.
How do you create an object?	ClassName obj = new ClassName(); — always use new.

How do you access a field?	obj.fieldName — using the dot operator.
How do you call a method?	obj.methodName() — using the dot operator.
What is null?	A reference that points to no object. Using it crashes with NullPointerException.
What is this?	A keyword inside a method referring to the current object. Used to resolve name conflicts.
What is garbage collection?	Java automatically frees memory of objects with no references. No delete needed.
What is a reference?	A variable holding the memory address of an object. Not the object itself.
What is encapsulation?	Making fields private and providing public methods to access/modify them safely.
What is a constructor?	A special method that runs automatically when an object is created with new.
Field vs local variable?	Fields: belong to object, live as long as object. Local: inside a method, disappear after.

Summary compiled from lecture slides — Classes and Objects in Java (Syeda Ajbina Nusrat, Lecturer, CSE, UITS) · Oracle Java Documentation